

MACHINE LEARNING NO SUPERVISADO

Extracción de paletas de color de obras de arte mediante clustering

Microproyecto 1 · Maestría en Inteligencia Artificial

Dataset: WikiArt (Kaggle, [steubk/wikiart](#))

Método: K-Means sobre píxeles en CIELAB, con k determinado por imagen

Entregable: notebook ejecutado y su export HTML

Equipo: bcollante98@gmail.com · saragallegovillada@gmail.com

Qué se pidió y cómo se organizó el trabajo

Construir, con técnicas de agrupación, un método que extraiga los **tonos dominantes** de una imagen y genere un muestrario de color representativo, más una visualización 2D de cómo se distribuye ese color.

Act.	Actividad de la rúbrica	Peso	Cómo se resolvió
(a)	Recolección de imágenes diversas, con justificación	10%	10 obras, 5 estilos, muestreo estratificado
(b)	Preparación de las imágenes, justificando decisiones	10%	RGB, resize a 150 px, CIELAB, aplanado
(c)	Pipeline de preparación reutilizable	10%	Función <code>preparar_imagen()</code>
(d)	Modelo de clustering con búsqueda de hiperparámetros y métrica justificada. El número de clusters debe variar por imagen	30%	K-Means, barrido $k = 2..12$, método del codo por imagen
(e)	Visualización 2D de la distribución de colores	—	t-SNE sobre los píxeles Lab
(f)	Función que convierte los clusters en un muestrario	15%	<code>generar_paleta()</code> y <code>mostrar_paleta()</code>
(g)	Evidencia sobre 4 o más imágenes de estilos distintos	25%	Muestra final con 5 estilos

EL REQUISITO CRÍTICO

La rúbrica exige explícitamente que **k se recalcule para cada imagen**. Un k fijo "que funcione bien en general" pierde el ítem de mayor peso, así que toda la actividad (d) se diseñó alrededor de esa restricción.

El dataset y cómo se accede a él

WikiArt (`steubk/wikiart` en Kaggle): unas 80.000 obras con metadatos en `data/classes.csv` : nombre de archivo, artista, género, dimensiones y partición.

La columna `genre` viene como una lista serializada; se parsea con `ast.literal_eval` y se toma el primer elemento como la columna `style` .

Elemento	Decisión
Metadatos	Versionados en el repo (archivo liviano)
Imágenes	Descarga bajo demanda con <code>kagglehub</code>
Caché	<code>~/.cache/kagglehub</code> , fuera del repo

POR QUÉ NO VERSIONAR LAS IMÁGENES

El dataset completo pesa decenas de gigabytes. Descargar solo las 10 obras seleccionadas mantiene el repositorio liviano y hace que el notebook sea reproducible en otra máquina sin mover archivos grandes: basta con credenciales de Kaggle.

EFFECTO SECUNDARIO ÚTIL

Como `kagglehub` guarda las imágenes en caché local, la segunda ejecución del notebook corre **sin red y sin credenciales**. Así se re-ejecutó de punta a punta antes de la entrega.

Selección del conjunto de imágenes

Se eligieron **5 estilos con paletas deliberadamente distintas** y 2 obras de cada uno: 10 imágenes, dentro del rango de 6 a 10 que pide la rúbrica y por encima del mínimo de 3 estilos.

Estilo	Identidad cromática buscada
Baroque	Claroscuro, marrones y dorados
Impressionism	Pastel, luminosa y difusa
Cubism	Tonos tierra y grises, geométrica
Ukiyo e	Colores planos y muy saturados
Action painting	Contrastes fuertes, primarios

POR QUÉ ESTOS ESTILOS Y NO OTROS

El objetivo del conjunto es **poner a prueba el método**, no representar la historia del arte. Estilos con paletas muy distintas obligan al algoritmo a comportarse de forma diferente en cada caso y hacen visible si *k* realmente se adapta.

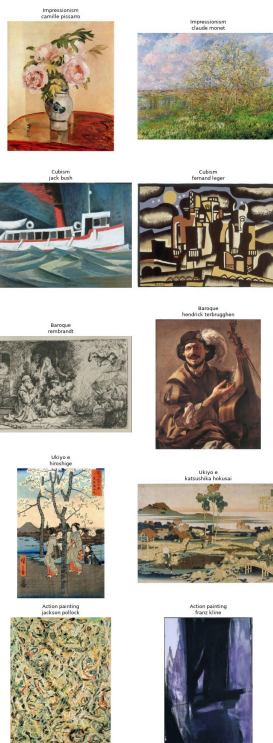
POR QUÉ SE EXCLUYÓ ROMANTICISM

Estaba en la lista inicial de 6 estilos, pero su paleta cálida y dramática **se solapa con la de Baroque**. Con un límite de 10 imágenes, gastar dos en un estilo redundante reduce la diversidad cromática efectiva.

POR QUÉ MUESTREO ESTRATIFICADO CON SEMILLA

`groupby("style").sample(n=2, random_state=42)` garantiza el mismo número de obras por estilo, sin sesgo hacia el estilo más frecuente, y hace la selección reproducible.

Las 10 obras seleccionadas



Dos obras por estilo. La diversidad cromática entre filas es la condición que hace informativo el resto del análisis.

El pipeline de preparación, en cuatro pasos

PASO 1

BGR a RGB

OpenCV lee en BGR. Se normaliza el orden de canales para que las figuras muestren los colores reales.

PASO 2

Resize del lado mayor a 150 px

Con `cv2.INTER_AREA`. Reduce de unos 2,5 millones de píxeles a unos 16.000.

PASO 3

RGB a CIELAB

Espacio perceptualmente uniforme: la distancia euclidiana se corresponde con la diferencia de color percibida.

PASO 4

Aplanado a matriz $n \times 3$

Cada píxel es una muestra independiente (L, a, b). Se descarta la posición (x, y).

POR QUÉ DESCARTAR LA POSICIÓN DEL PÍXEL

El objetivo es agrupar **colores**, no regiones de la imagen. Incluir (x, y) haría que el modelo segmentara zonas espaciales, que es un problema distinto.

POR QUÉ ENCAPSULAR EN UNA FUNCIÓN — ACTIVIDAD (C)

`preparar_imagen(path, max_dim=150)` devuelve la imagen RGB y los píxeles Lab. Es aplicable a **cualquier** imagen del dataset, no solo a las 10 elegidas, y es el único punto que hay que tocar si cambia el espacio de color o el resize.

Por qué CIELAB y no RGB

K-Means y MeanShift agrupan **minimizando distancia euclidiana**. La pregunta es en qué espacio medir esa distancia.

En sRGB, dos pares de colores separados por la misma distancia numérica pueden verse casi idénticos o completamente distintos: el espacio no es perceptualmente uniforme.

CIELAB fue diseñado para que la distancia euclidiana aproxime la diferencia percibida. Es exactamente la definición de diferencia de color CIE76:

$$\Delta E^*_{ab} = \sqrt{(\Delta L^*)^2 + (\Delta a^*)^2 + (\Delta b^*)^2}$$

L^* es luminosidad, a^* el eje verde-rojo, b^* el eje azul-amarillo

LA CONSECUENCIA PRÁCTICA

Agrupar en Lab produce clusters que se corresponden con **"colores que una persona agruparía"**. Agrupar en RGB produce clusters matemáticamente válidos pero perceptualmente arbitrarios, y una paleta que no convence al ojo.

Detalle de implementación. OpenCV, sobre arreglos de 8 bits, no entrega Lab en su rango canónico sino escalado a byte:

$$L_8 = 255/100 \cdot L^* \quad a_8 = a^* + 128 \quad b_8 = b^* + 128$$

Los tres canales quedan en 0 a 255, con 128 como neutro en a y b

Ese detalle importa porque es lo que hace que los tres canales **compartan rango**, y es la base de la siguiente decisión.

Por qué reducir a 150 px, y por qué eso no daña la paleta

El clustering de color **no necesita resolución espacial**: solo necesita la distribución de colores, que se conserva al submuestrear.

2,5 M

píxeles por obra
en el original

~16 K

píxeles tras
la preparación

Esa reducción de dos órdenes de magnitud es lo que hace **viable en tiempo** ajustar 11 modelos de K-Means por imagen en la actividad (d).

POR QUÉ INTER_AREA Y NO BICÚBICA

INTER_AREA promedia el área fuente de cada píxel destino, así que todo color resultante es una combinación de colores que ya existían. Las interpolaciones bicúbica o Lanczos tienen lóbulos negativos y pueden generar **colores que no estaban en la obra**, contaminando la paleta.

Evidencia cuantitativa. Estadísticos RGB antes y después del resize, para la primera obra del conjunto:

	R	G	B
Original (2,46 M px)	168,0 ± 66,2	159,7 ± 61,5	119,5 ± 56,8
Reducida (17,4 K px)	168,0 ± 48,8	159,7 ± 43,2	119,5 ± 38,1

CÓMO SE LEE ESTA TABLA

La **media por canal se conserva exactamente**: la tendencia central de cada color no se mueve. La desviación estándar baja porque el promediado atenúa la textura fina de pincelada.

Eso no perjudica el objetivo: los centros de los clusters dependen de **dónde se concentra la masa de píxeles**, no de la varianza local. Si acaso, reduce ruido.

Comprobación visual del redimensionado

Action painting - jackson pollock

Original 1382x1782 (2462724 px)



Redimensionada 116x150 (17400 px)



La composición de color se mantiene: lo que se pierde es detalle espacial, que el método no usa.

Por qué no se estandariza ni se normaliza

Estandarizar variables antes de un algoritmo basado en distancias es la práctica por defecto. Aquí se decidió **no hacerlo**, y conviene justificarlo porque va contra la costumbre.

RAZÓN 1 — NO HACE FALTA

El motivo habitual para estandarizar es que una variable con rango mucho mayor domine la distancia. En Lab de 8 bits los tres canales ya viven en **0 a 255** y tienen significado perceptual homogéneo: ninguno domina artificialmente.

RAZÓN 2 — SERÍA CONTRAPRODUCENTE

Aplicar z-score por canal reescalaría ΔL , Δa y Δb por factores distintos y **rompería la equivalencia con ΔE^*_{ab}** . Es decir, destruiría justamente la propiedad por la que se eligió Lab.

$$x' = (x - \mu) / \sigma$$

Transformación descartada: deforma las distancias perceptuales

RAZÓN 3 — ESCALAR A [0,1] TAMPOCO APORTA

Una normalización min-max dividiría los tres canales por el mismo factor. Es inocua, pero no cambia nada: el rango ya es común. Añadir un paso que no altera el resultado solo agrega superficie de error.

Conclusión: los píxeles Lab se pasan al modelo tal cual salen del pipeline. La decisión de no transformar es tan deliberada como las de transformar.

Por qué K-Means como modelo principal

Para cada imagen se ajusta K-Means sobre sus píxeles Lab, minimizando la inercia:

$$J(k) = \sum_j \sum_{x \in C_j} ||x - \mu_j||^2$$

Suma de distancias al cuadrado dentro de cada cluster

Hiperparámetro	Valor
Inicialización	<code>k-means++</code>
Reinicios	<code>n_init = 10</code>
Semilla	<code>random_state = 42</code>
Rango de búsqueda	<code>k = 2 a 12</code>

POR QUÉ K-MEANS Y NO OTRO ALGORITMO

Su **único hiperparámetro relevante es k**. Eso convierte la exigencia de la rúbrica en una búsqueda limpia y auditable sobre una sola dimensión, en lugar de una exploración difusa de varios parámetros acoplados.

POR QUÉ K-MEANS++ CON 10 REINICIOS

K-Means converge a óptimos locales que dependen de la inicialización. `k-means++` separa los centros iniciales y los 10 reinicios reducen el riesgo de que un mal arranque distorsione la curva de inercia, que es de donde se decide k.

COSTO

11 valores de k por 10 imágenes = **110 ajustes**, más 10 reajustes finales. Es asumible porque K-Means es lineal en el número de píxeles y el pipeline ya los redujo a unos 16.000.

Cómo se elige k para cada imagen: el método del codo

El codo es el punto de **rendimientos decrecientes**: el k a partir del cual añadir un cluster ya casi no reduce la distorsión. Para localizarlo sin inspección visual se aplica el método de **Kneedle**:

- Se normalizan ambos ejes de la curva de inercia a [0, 1].
- Se traza la recta que une el primer y el último punto.
- El codo es el k con **máxima distancia vertical** a esa recta.

$$k^* = \operatorname{argmax}_i [L(\hat{x}_i) - \hat{y}_i]$$

Como la curva es convexa y decreciente, queda bajo la cuerda y la diferencia es positiva

POR QUÉ EL CODO ES EL CRITERIO CORRECTO AQUÍ

Una paleta útil es la **más pequeña que ya captura la variación de color** de la obra. Eso es literalmente lo que busca el codo. Un criterio que premie más separación daría pocos colores; uno que premie más detalle daría docenas de tonos casi iguales.

POR QUÉ KNEEDLE Y NO MIRAR LA GRÁFICA

Elegir el codo "a ojo" no es reproducible ni escalable a 10 imágenes, y sería imposible defenderlo como búsqueda de hiperparámetros. Kneedle lo convierte en una **regla determinista** que se aplica igual a toda imagen nueva.

ESTO ES LO QUE CUMPLE EL REQUISITO CRÍTICO

La regla se ejecuta **por imagen**, sobre la curva de inercia de esa imagen. No hay ningún k global.

Por qué el silhouette se probó y se descartó como criterio

El silhouette mide cohesión frente a separación, acotado entre -1 y 1 :

$$s(i) = [b(i) - a(i)] / \max\{ a(i), b(i) \}$$

$a(i)$: distancia media dentro del cluster. $b(i)$: distancia media al cluster vecino más cercano

Era el candidato natural como criterio primario. Se implementó, se evaluó y **se descartó**, por una razón concreta.

QUÉ PASÓ

El silhouette se maximiza en **$k = 2$ en 9 de las 10 imágenes**.

Premia particiones gruesas y muy separadas, típicamente el eje claro-oscuro de la obra, no la granularidad cromática que necesita una paleta.

POR QUÉ SE CONSERVA DE TODOS MODOS

Como **control**. En el k que elige el codo, el silhouette va de **0,38 a 0,58**: claramente positivo. Eso confirma que la partición elegida tiene estructura real y no está partiendo grupos homogéneos de forma arbitraria.

POR QUÉ SE CALCULA SOBRE UNA SUBMUESTRA

El silhouette es $O(n^2)$. Se evalúa sobre 2.000 píxeles por imagen; como solo se usa para verificar el orden de magnitud, la submuestra es suficiente.

LECTURA PARA LA SUSTENTACIÓN

Reportar una métrica que no se terminó usando, y explicar por qué, es parte del resultado: la decisión de qué criterio gobierna k está documentada con evidencia, no asumida.

MeanShift: una segunda opinión que no recibe k

MeanShift **deduce** el número de clusters a partir de los modos de la densidad, desplazando cada punto hacia la media local ponderada por un núcleo de ancho *bandwidth*.

El `bandwidth` se estima **por imagen** con `estimate_bandwidth(quantile=0.2)`, y corre sobre una submuestra de 3.000 píxeles.

POR QUÉ INCLUIRLO

Si un algoritmo que **nunca ve el valor de k** encuentra un número de grupos del mismo orden que el codo, refuerza que ese k describe estructura real de los datos y no es un artefacto de K-Means.

QUÉ DIO

Entre 2 y 6 clusters en **9 de 10 imágenes**, en el mismo orden de magnitud que el codo (4 o 5). El contraste sostiene la elección.

LA EXCEPCIÓN, REPORTADA

En el Pollock *Eyes in the Heat* el bandwidth estimado (24,47) resulta demasiado grande y MeanShift **colapsa todo en 1 cluster**. Es la conocida hipersensibilidad del método al bandwidth.

Es exactamente la razón por la que MeanShift es contraste y no método principal: su hiperparámetro es continuo y difícil de fijar de forma robusta, mientras que el k de K-Means es discreto y acotado.

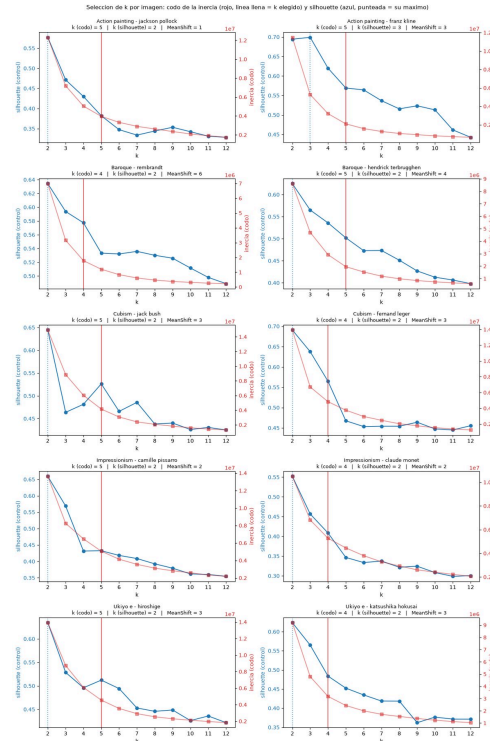
k se recalcula para cada imagen

Estilo y obra	k (codo)	k (silhouette)	silhouette en el k elegido	MeanShift	bandwidth
Action painting — Pollock	5	2	0,382	1	24,47
Action painting — Kline	5	3	0,569	3	21,21
Baroque — Rembrandt	4	2	0,577	6	15,75
Baroque — Terbrugghen	5	2	0,502	4	15,90
Cubism — Jack Bush	5	2	0,526	3	29,99
Cubism — Léger	4	2	0,565	3	24,05
Impressionism — Pissarro	5	2	0,433	2	26,94
Impressionism — Monet	4	2	0,409	2	23,49
Ukiyo e — Hiroshige	5	2	0,512	3	28,14
Ukiyo e — Hokusai	4	2	0,484	3	21,87

LECTURA HONESTA DEL RANGO

k va de **4 a 5**: varía por imagen, como exige la rúbrica, pero el rango es estrecho. A 150 px estas diez obras tienen una complejidad cromática efectiva parecida y la curva de inercia baja de forma suave. El mismo procedimiento sobre un dibujo casi monocromo elegiría un k menor.

Las curvas de las que se decide k



Rojo: inercia, con línea llena en el k del codo (criterio). Azul: silhouette, con puntuada en su máximo (control), casi siempre en k = 2.

Visualización 2D: por qué t-SNE y no PCA

Cada imagen preparada es una nube de píxeles en 3D (Lab). Para ver cómo se distribuye el color y cómo lo particiona K-Means, hay que proyectarla a 2D.

POR QUÉ T-SNE

Preserva la estructura **local**: píxeles de color parecido quedan cerca y los grupos densos se separan visualmente. Es justo lo que se quiere mostrar.

POR QUÉ NO PCA

PCA es una proyección lineal sobre las dos direcciones de mayor varianza: más fiel a distancias globales, pero **tiende a solapar grupos** que en Lab sí están separados.

Decisión	Valor y motivo
Submuestreo	3.000 píxeles: t-SNE es $O(n^2)$
<code>perplexity</code>	30: número efectivo de vecinos
<code>init</code>	<code>"pca"</code> : embedding estable
Color del punto	Su color RGB real

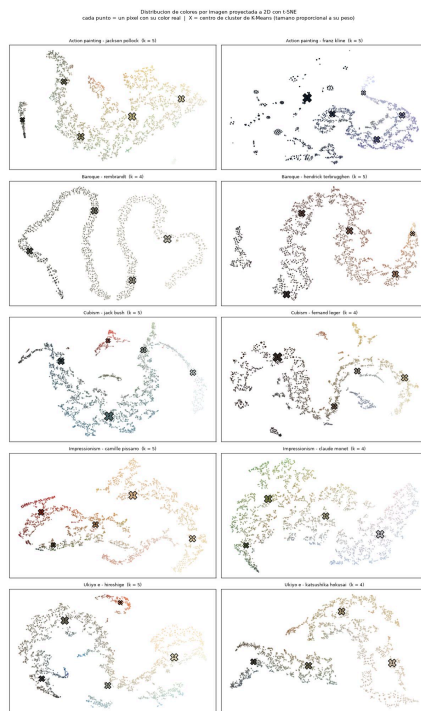
POR QUÉ PINTAR CADA PUNTO CON SU COLOR REAL

Como K-Means agrupa **en el espacio de color**, cada cluster ya es una región cromática coherente. Pintar por color real muestra a la vez la distribución y la partición; una paleta categórica arbitraria solo mostraría la segunda.

DETALLE TÉCNICO

t-SNE no tiene `transform`, así que no se pueden proyectar los centros sobre un embedding ya ajustado. Se corre **una sola vez sobre la pila** [píxeles ; centros] y luego se separan.

La nube de color de cada obra



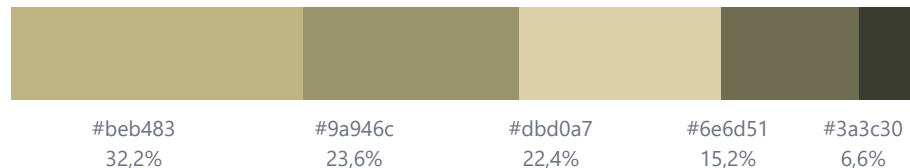
Cada punto es un píxel con su color real. Las X son los centros de K-Means, con tamaño proporcional a su peso. Advertencia: en t-SNE solo la vecindad es fiable, no las distancias absolutas.

La función que convierte clusters en un muestrario

`generar_paleta(centros_lab, proporciones)` toma **exactamente la salida de la actividad (d)** y no recalcula nada: los centros ya son, por construcción, los colores dominantes.

Cada centro se pasa de Lab a sRGB y a código hexadecimal. El peso de cada color es la fracción de píxeles asignada a su cluster.

Salida real para el Pollock (k = 5):



POR QUÉ DEVUELVE UN DATAFRAME Y NO SOLO UN GRÁFICO

La salida es una tabla con `rank`, `hex`, `R`, `G`, `B`, `proporcion`. Así la paleta queda como un **dato reutilizable**, no atrapada dentro de una figura. El caso de uso es un diseñador que copia el código hex.

POR QUÉ ORDENAR POR PROPORCIÓN

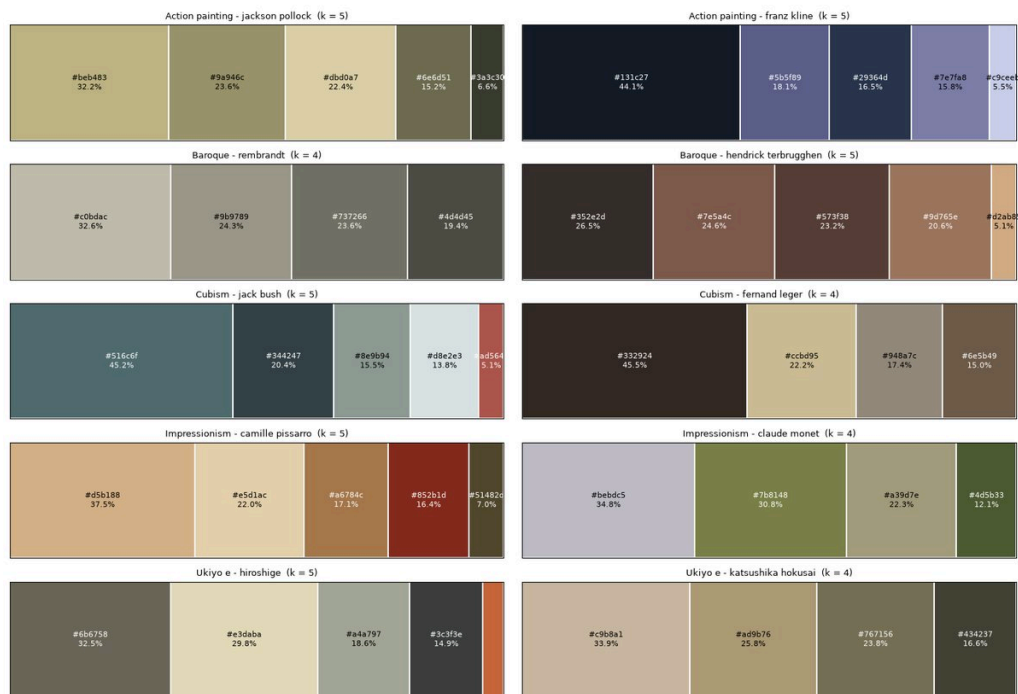
El primer color del muestrario es el más presente en la obra. La columna `rank` hace explícito ese orden.

POR QUÉ FRANJAS DE ANCHO PROPORCIONAL

Un muestrario de franjas iguales dice *qué* colores hay; uno de ancho proporcional dice además **cuánto pesa cada uno**. La diferencia entre un color dominante y uno de acento es información, y es gratis mostrarla.

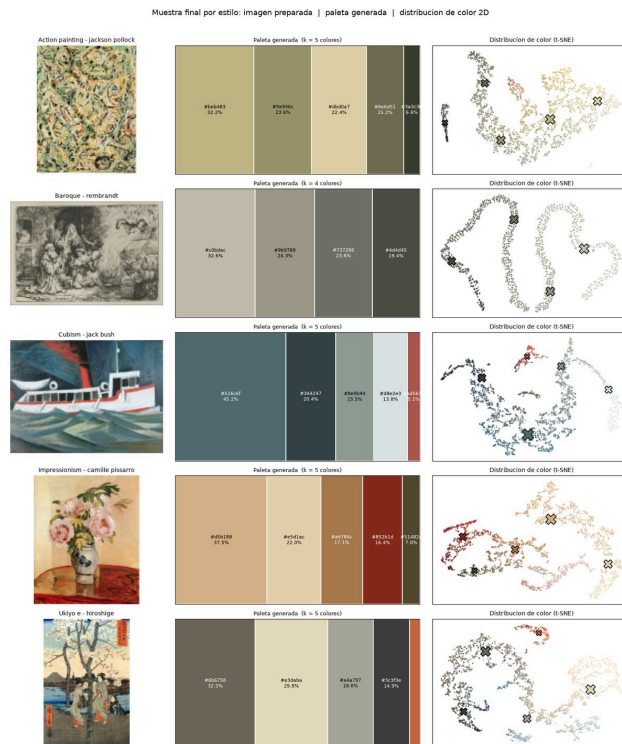
Las 10 paletas generadas

Paleta generada por imagen: cada franja es un color dominante; su ancho es su proporcion en la imagen



Cada franja es un color dominante y su ancho es su proporción en la obra. El número de franjas cambia entre imágenes porque k cambia.

Muestra final: el método completo sobre 5 estilos



Un representante por estilo, por encima del mínimo de 4 que pide la rúbrica. Por fila: imagen preparada, paleta generada y distribución de color 2D.

Qué demuestra la muestra final

EL MÉTODO NO INVENTA COLOR

El grabado de Rembrandt es prácticamente monocromo y produce un muestrario de **cuatro tonos casi neutros**, una rampa de grises cálidos. No aparecen colores que no están en la obra.

CAPTURA LOS ACENTOS, NO SOLO LO DOMINANTE

Encuentra los rojos que el ojo busca primero: el rojo oscuro `#852b1d` del mantel del Pissarro (16,4%) y el `#ad5647` del casco del ferry en el Bush, que pesa solo **5,1%** pero queda bien separado en la nube 2D.

LAS TRES VISTAS SON CONSISTENTES

El color y el ancho de cada franja se corresponden con la posición y el tamaño de la X del mismo color en la nube t-SNE, y con lo que se ve en la imagen. El muestrario es, en esencia, **la nube 2D resumida en una barra ordenada**.

EL NÚMERO DE COLORES SE ADAPTA

Las obras mostradas usan $k = 4$ o 5 , decidido por imagen en la actividad (d). Es la evidencia visual de que se cumple el requisito de mayor peso.

Limitaciones y trabajo futuro

RANGO DE K ESTRECHO

k varía por imagen pero solo entre 4 y 5. Un conjunto con casos más extremos (un dibujo a lápiz frente a un vitral) separaría más los valores y haría más visible la adaptación.

MEANSHIFT FRÁGIL

Colapsó a 1 cluster en una obra. Buscar el `quantile` por imagen, o usar otro estimador de densidad, lo haría un contraste más robusto.

MÉTRICA ALTERNATIVA

En lugar del silhouette, un criterio perceptual explícito: exigir que la ΔE^*_{ab} mínima entre centros supere el umbral de distinguibilidad, unas 2,3 unidades en CIE76.

SUPUESTO DE K-MEANS

Asume clusters isotrópicos de varianza similar. Las distribuciones de color reales suelen ser alargadas (rampas de iluminación). Un GMM con covarianza completa las capturaría mejor, a costa de más parámetros.

INFORMACIÓN ESPACIAL DESCARTADA

Incluir (x, y) ponderado permitiría segmentar regiones además de colores, pero cambiaría el objetivo del proyecto.

CUANTIZACIÓN DE 8 BITS

La conversión Lab de OpenCV cuantiza a^* y b^* en pasos de 1 unidad. Para paletas de precisión alta convendría trabajar en `float32` con Lab sin escalar.

Reproducibilidad y entregables

TODA LA LÓGICA VIVE EN EL NOTEBOOK

No hay módulos `.py` auxiliares. El entregable son dos archivos y un corrector solo abre esos dos: si parte del código viviera fuera, el trabajo no sería evaluable.

ENTORNO FIJADO

Entorno virtual propio, `requirements.txt` con cotas de versión, y un kernel de Jupyter con nombre (`Python (Proyecto1MLNS)`) que apunta a ese entorno. Elimina la ambigüedad de ejecutar el notebook contra otro intérprete.

SEMILLA ÚNICA EN TODO EL PIPELINE

`random_state = 42` en el muestreo de imágenes, k-means++, las submuestras de silhouette y MeanShift, y la inicialización de t-SNE. Una re-ejecución completa reproduce las mismas tablas y figuras.

EJECUTADO DE PUNTA A PUNTA

El notebook se re-ejecutó completo con `nbconvert --execute` antes de entregar: las 19 celdas de código tienen su output embebido, numeradas 1 a 19, sin errores. De ahí se exportó el HTML.

Resumen de la sustentación

Decisión	Qué se hizo	Por qué
Conjunto de datos	10 obras, 5 estilos contrastantes	Poner a prueba el método sobre paletas muy distintas
Espacio de color	CIELAB	La distancia euclidiana aproxima la diferencia percibida
Resize	150 px con INTER_AREA	Viabilidad computacional sin inventar colores; media por canal intacta
Escalado	Ninguno	Los canales ya comparten rango; estandarizar rompería la uniformidad perceptual
Modelo	K-Means, $k = 2..12$ por imagen	Un solo hiperparámetro: búsqueda limpia y auditable
Criterio de k	Método del codo (Kneedle)	Busca la paleta más pequeña que ya captura la variación de color
Silhouette	Control, no criterio	Colapsa a $k = 2$; confirma estructura pero no fija la granularidad
MeanShift	Contraste independiente	Valida el orden de magnitud de k sin recibirlo
Visualización	t-SNE, puntos con su color real	Preserva estructura local; muestra distribución y partición a la vez
Paleta	DataFrame y franjas proporcionales	Dato reutilizable, y el ancho comunica el peso de cada color

LA CONCLUSIÓN

El método entrega una paleta interpretable por obra, con **k decidido individualmente para cada imagen** mediante una regla determinista y reproducible, verificada con dos señales independientes y evidenciada sobre 5 estilos distintos.